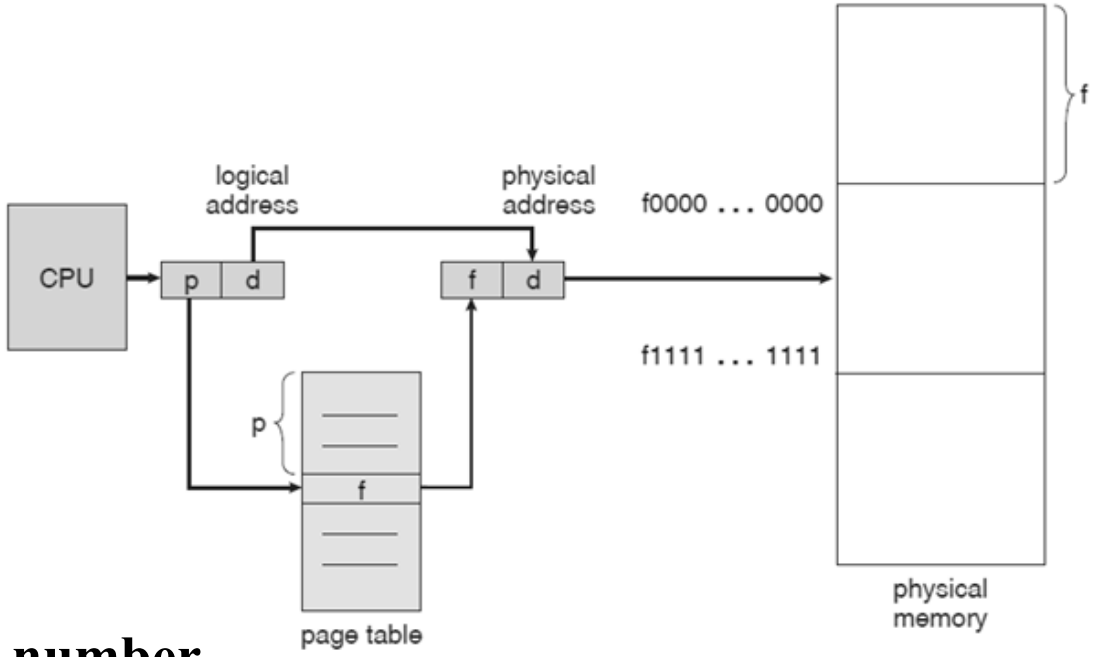
Virtual Memory

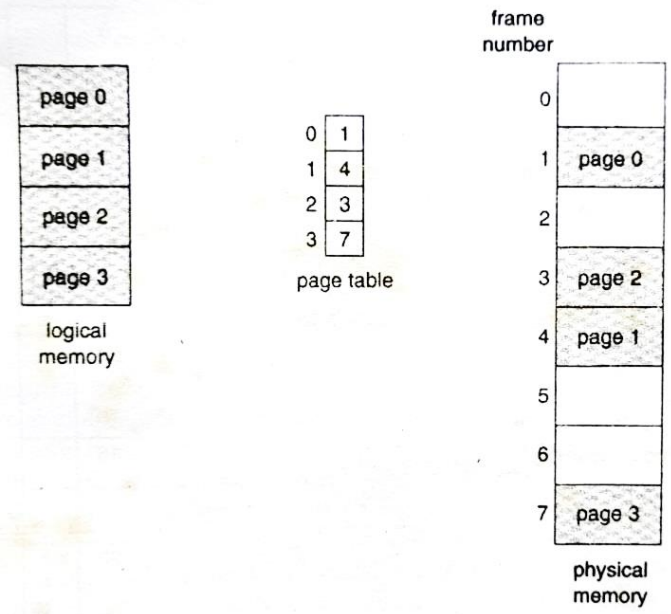
Virtual Memory

- Virtual memory is **a storage allocation technique**, where programmer will think that **whole program is residing in main memory**, though the **size of main memory is smaller than that of the program.**
- Virtual memory provides each program, **a part of the main memory and stores the rest of its code and data on a hard disk**.
- Whenever required **the old data will be swapped out from main memory**, and the **new data will be swapped in from secondary storage**. Swap in and swap out are also called Roll In and Roll Out.
- The **Memory Management Unit (MMU)** transfers the currently needed part of the program from secondary memory to the main memory for execution.
- The **address of virtual memory**, generated by the CPU is referred to as **Logical Address**.
- The **address of main memory** is called the **Physical Address**.
- **MMU converts the logical address into physical address**.
- In **logical address space**, i.e. in hard disk, **multiple processes are located**, and **each process is divided into number of pages**.
- **Paging and Segmentation** are processes to swap in and swap out data between main memory and secondary memory.

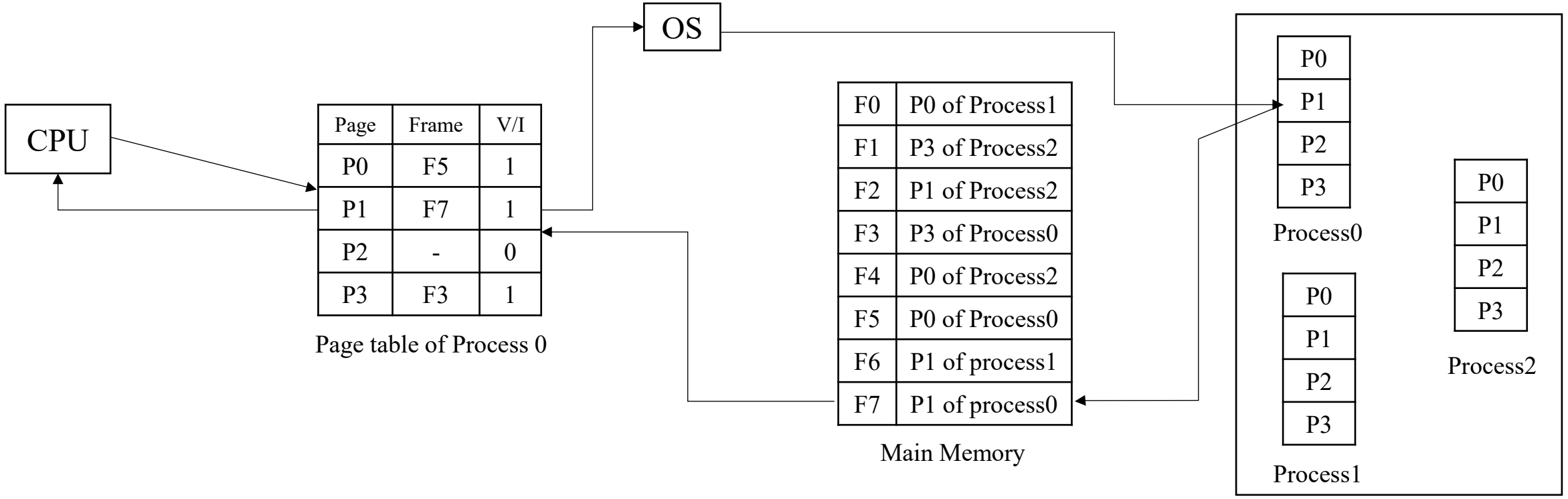
Paging

- Processes in the **secondary storage**, (Logical memory) are divided into equal size **pages**.
- The **main memory** (Physical memory) will also be divided into equal-fixed size chunks called **frames**.
- **Page size is equal to the frame size**.
- When a process is to be executed, its **pages from the logical memory is loaded into any available frame of the physical memory**.
- **Logical address** generated by the CPU is **divided into page number (p) and page offset/page size (d)**.
- **Each process typically requires its own page table** to ensure process isolation and allow for efficient memory management.
- **Page table contains the page number** and its **corresponding frame number** or base address of the main memory.
- **Base address** is combined with the **page offset** to define the **physical memory address.**
- So **physical address is divided into frame number and frame size**.





Virtual Memory



1. CPU is looking for page 1 of process 0. MMU will search in the page table. If found, will send the page to CPU, otherwise page fault will occur.
2. If page fault occur control will transfer to the OS
3. OS will search the page in hard disk.
4. Will copy that page into available frame of main memory.
5. Update the page table.
6. Send data to CPU

❖ Effective Memory Access Time (EMAT)=
P * page fault service time + (1-P) * Main memory access time

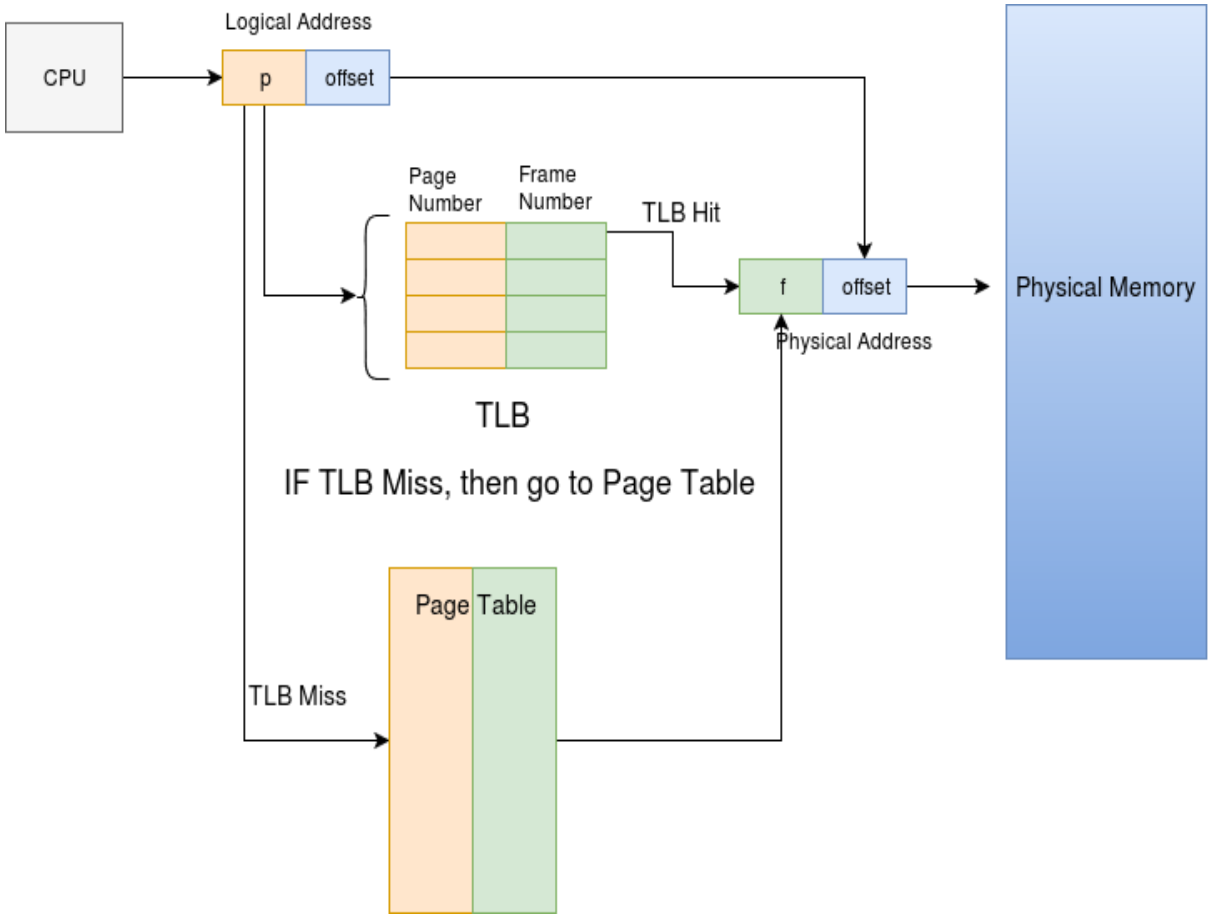
P→ Page Fault Rate

Numerical

- Logical address space = 4GB, Physical address space=64MB. Page size=4KB. Find no of pages, no of frames and no of entries in page table.
- Consider a system which has Logical address = 7bits, Physical address = 6bits, page size = 8 words. Then calculate No of pages and No of frames.

Translation Lookaside Buffer (TLB)

- Every memory access by a program take **at least twice as long**:
  (i) memory access **to obtain the physical address**.
  (ii) memory access **to get the data**.
- A **part of page table** that **stores inside the cache memory** is called TLB.
- TLB **keeps track of recently used pages to avoid an access to the page table**.
- TLB **acts as a cache of the page table** for the entries that map to physical pages only. **Contains a subset of the virtual-to-physical page mappings** that are in the page table.



- If found (**TLB Hit**), its frame number is available
- If not found (**TLB Miss**), page number is searched in the page table of main memory, and frame number is obtained.
- The **new entries are added to the TLB for future reference**.
- By default, **TLB uses fully associative mapping technique.** If TLB is full, **LRU is used to replace TLB entries**.

TLB Hit and TLB Miss

Steps in TLB hit:

i. CPU generates virtual (logical) address.

ii. It is checked in TLB (present).

iii. Corresponding frame number is retrieved, which now tells where the main memory page lies.

Steps in TLB miss:

i. CPU generates virtual (logical) address.

ii. It is checked in TLB (not present).

iii. Now the page number is matched to page table residing in main memory (assuming no page fault).

iv. Corresponding frame number is retrieved, which now tells where the main memory page lies.

v. The TLB is updated with new Page Table Entry (PTE) (if space is not there, one of the replacement technique comes into picture i.e either FIFO, LRU or MFU etc).

Effective memory access time (EMAT)

Effective memory access time(EMAT) : TLB is used to reduce effective memory access time as it is a high speed associative cache.

EMAT = h*(c+m) + (1-h)*(c+2m)
where, h = hit ratio of TLB
m = Memory access time
c = TLB access time

Problem 1: In a paging scheme using TLB, TLB access time 10ns and Main memory access time 50ns. What is the effective memory access time, if TLB hit ratio is 90% and there is no page fault.

Ans: EMAT = h*(c+m) + (1-h)*(c+2m)
h =90% = 0.9
c = 10ns
m =50ns
EMAT = 0.9 (10 +50) + 0.1 (10+50+50)
= 54 + 11 = 65ns

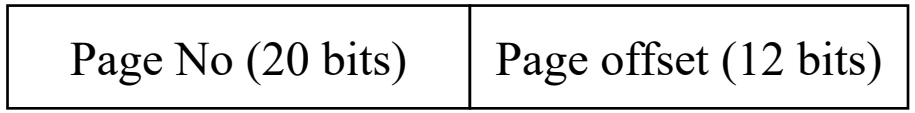
Effective memory access time (EMAT)

Problem 2: A CPU generates 32 bit virtual address. The page size is 4KB. The processor has a TLB which can hold a total of 128 page table entries and is a 4-way set associative mapping. What will be the minimum size of the TLB tag field?

Ans: Virtual address/Logical address = 32bit

Page size = 4KB $= 2^2 \times 2^{10} = 2^{12}$

So 12 bits for page offset and (32-12) 20 bits for page number.



Logical Address (32 bits)

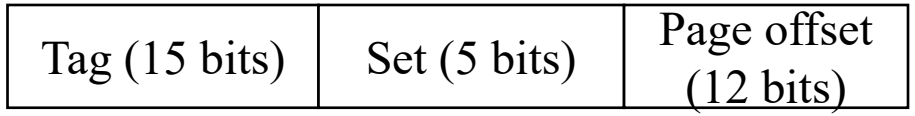
Page table entries = No of lines in page table = 128 = 2^7

No of sets = No of lines / 4 way set associative = 128 / 4 = 2^7 / $2^2 = 2^5$

Page offset 12 bits.

Logical address = Physical address

So, (32-(5+12)) = 15 bits for tag field



Physical Address (32 bits)

Segmentation

